Relatório Problema da Mochila 0/1

João Pedro Neves e Juan Pablo Tomba

10 de outubro de 2025

1 Introdução

O Problema da Mochila 0/1 é um dos problemas clássicos de otimização combinatória. O desafio consiste em escolher, dentre um conjunto de itens com pesos e valores associados, aqueles que devem ser colocados em uma mochila de capacidade limitada, de modo a maximizar o valor total transportado sem ultrapassar o peso máximo permitido. O nome "0/1" reflete a restrição de que cada item pode ser incluído ou excluído da mochila — não é permitido fracionar itens.

O objetivo deste trabalho é implementar e analisar três abordagens distintas para resolver o problema: Força Bruta, Algoritmo Guloso e Programação Dinâmica. Além de comparar suas implementações e eficiências teóricas, o projeto inclui uma análise empírica com medições reais de tempo de execução para diferentes tamanhos de entrada, buscando observar na prática como as diferenças teóricas se manifestam no desempenho.

2 Implementações

2.1 Brute Force

```c
#include <stdio.h>
#include <stdlib.h>

// struct It para organização dos valores de cada item (nome, peso, valor e razao)
typedef struct {
    int peso;
    int valor;
    char nome[50];
} It;

// funcao recursiva que encontra todos os subconjuntos possíveis de elementos, e escolhe o de melhor resultado
void encontrar_melhor_combinacao(
    It todos_os_itens[],   //recebe todos os itens que serão avaliados
    int n_total_itens,     //numero total de itens
    It combinacao_atual[], //combinacao atual para analisar a cada chamada da recursao
    int tam_comb_atual,    //numero de itens na combinacao atual
    int indice,            //qual o nivel atual da recursao (tamanho do subconjunto)
    int capacidade_mochila, //capacidade total da mochila
```

```c
    int* maior_valor_global, //maior valor que foi encontrado pela melhor combinacao ate o momento
    It** melhor_combinacao_global, //conjunto de itens da melhor combinacao ate o momento
    int* tamanho_melhor_combinacao //numero de itens da melhor combinacao ate o momento

) {

    //condicao de parada: tamanho do subconjunto ser igual ao numero total de itens
    if (indice == n_total_itens) {
        int peso_atual = 0;
        int valor_atual = 0;

        //analisa a combinacao formada nessa etapa da recursao e soma os pesos e os valores dos itens que ela contem
        for (int i = 0; i < tam_comb_atual; i++) {
            peso_atual += combinacao_atual[i].peso;
            valor_atual += combinacao_atual[i].valor;
        }

        //se o peso total da combinacao atual for menor que a capacidade da mochila e o valor dos itens for maior que o melhor ate entao
        //atualiza o maior valor encontrado e passa os itens da combinacao atual para o vetor melhor combinacao
        if (peso_atual <= capacidade_mochila && valor_atual > *maior_valor_global) {
            *maior_valor_global = valor_atual;
            *tamanho_melhor_combinacao = tam_comb_atual;

            *melhor_combinacao_global = (It*) realloc(*melhor_combinacao_global, tam_comb_atual * sizeof(It));

            for (int i = 0; i < tam_comb_atual; i++) {
                (*melhor_combinacao_global)[i] = combinacao_atual[i];
            }
        }

        return;
    }

    //Escolhe que o item nao vai entrar no conjunto, e chama a recursao para o proximo item
    encontrar_melhor_combinacao(todos_os_itens, n_total_itens, combinacao_atual, tam_comb_atual, indice + 1, capacidade_mochila, maior_valor_global, melhor_combinacao_global, tamanho_melhor_combinacao);

    //Escolhe que o item vai entrar no conjunto, e chama a recursao para o proximo item
    combinacao_atual[tam_comb_atual] = todos_os_itens[indice];
    encontrar_melhor_combinacao(todos_os_itens, n_total_itens, combinacao_atual, tam_comb_atual + 1, indice + 1, capacidade_mochila, maior_valor_global, melhor_combinacao_global, tamanho_melhor_combinacao);
```

```c
}

int main() {
    //Trata a entrada dos valores de capacidade da mochila e quantidade de itens
    printf("Esse programa resolve o problema da mochila (knapsack problem) atraves da forca bruta.\n\n");

    int w;
    printf("Informe o peso maximo suportado pela mochila: ");
    scanf("%d", &w);

    int n;
    printf("Digite quantos itens serao avaliados: ");
    scanf("%d", &n);

    //cria vetor de itens dinamicamente
    It* itens = (It*) malloc(n * sizeof(It));
    if (itens == NULL) {
        printf("Erro de alocacao de memoria!\n");
        return 1;
    }

    //pega as entradas do usuario
    printf("Digite os %d itens no formato\n\"nome\" \"peso\" \"valor\":\n", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%s %d %d", itens[i].nome, &itens[i].peso, &itens[i].valor) != 3) {
            printf("Entrada invalida.\n");
            free(itens);
            return 1;
        }
    }

    //cria a combinacao atual que sera analisada pela funcao recursiva
    It* combinacao_atual = (It*) malloc(n * sizeof(It));
    if (combinacao_atual == NULL) {
        printf("Erro de alocacao de memoria!\n");
        free(itens);
        return 1;
    }

    //inicia as variaveis da melhor combinacao
    int maior_valor_global = 0;
    It* melhor_combinacao_global = NULL;
    int tamanho_melhor_combinacao = 0;


    encontrar_melhor_combinacao(itens, n, combinacao_atual, 0, 0, w, &maior_valor_global, &melhor_combinacao_global, &tamanho_melhor_combinacao);

    //se encontrou uma combinacao com valor válido, retorna os itens, o valor total e o peso total dessa combinacao
    if (maior_valor_global > 0) {
```

```c
        printf("\n--- Solucao Otima Encontrada ---\n");
        printf("Maior valor: %d\n", maior_valor_global);
        printf("Itens na mochila:\n");
        int peso_final = 0;
        for (int i = 0; i < tamanho_melhor_combinacao; i++) {
            printf("  - %s (Peso: %d, Valor: %d)\n", melhor_combinacao_global[i].nome, melhor_combinacao_global[i].peso, melhor_combinacao_global[i].valor);
            peso_final += melhor_combinacao_global[i].peso;
        }
        printf("Peso total na mochila: %d (Capacidade: %d)\n", peso_final, w);
    } else {
        printf("\nNenhuma combinacao de itens coube na mochila.\n");
    }

    //libera vetores alocados dinamicamente
    free(itens);
    free(combinacao_atual);
    free(melhor_combinacao_global);

    return 0;
}
```

2.2 Guloso

```c
#include <stdio.h>

// struct item para organização dos valores de cada item (nome, peso, valor e razao)
typedef struct {
    char nome[50];
    int peso;
    int valor;
    double razao;
} Item;

// função auxiliar para a troca de itens no quicksort
void swap(Item *a, Item *b) {
    Item temp = *a;
    *a = *b;
    *b = temp;
}

// quick sort decrescente, que organiza os itens de acordo com sua razao de: (valor / peso)
void quicksort(Item v[], int inicio, int fim) {
    int i = inicio, j = fim;
    double pivo = v[(inicio + fim) / 2].razao;

    while (i <= j) {
        while (v[i].razao > pivo) i++;
        while (v[j].razao < pivo) j--;
        if (i <= j) {
```

```c
            swap(&v[i], &v[j]);
            i++;
            j--;
        }
    }

    if (inicio < j) quicksort(v, inicio, j);
    if (i < fim) quicksort(v, i, fim);
}

int main() {
    printf("Esse programa resolve o problema da mochila (knapsack problem) atraves do metodo guloso.\n\n");

    int n, capacidade;
    printf("Informe o peso maximo suportado pela mochila: ");
    scanf("%d", &capacidade);

    printf("Digite quantos itens serao avaliados: ");
    scanf("%d", &n);

    Item itens[n];
    // leitura dos itens da mochila
    printf("Digite os %d itens no formato\n\"nome\" \"peso\" \"valor\":\n", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%s %d %d", itens[i].nome, &itens[i].peso, &itens[i].valor) != 3) {
            printf("Entrada invalida.\n");
            return 1;
        }
        itens[i].razao = (double) itens[i].valor / itens[i].peso;
    }

    quicksort(itens, 0, n - 1);

    int peso_atual = 0;
    int valor_total = 0;

    printf("\n--- Itens Escolhidos pela Estrategia Gulosa ---\n");
    printf("Itens na mochila:\n");

    // vai inserindo os itens com maior razão valor/peso, até não ter espaço na mochila para o próximo
    for (int i = 0; i < n; i++) {
        if (peso_atual + itens[i].peso <= capacidade) {
            peso_atual += itens[i].peso;
            valor_total += itens[i].valor;
            printf("  - %s (Peso: %d, Valor: %d, Razao: %.2f)\n", itens[i].nome, itens[i].peso, itens[i].valor, itens[i].razao);
        }
    }

    if (valor_total == 0){
        printf("\nNenhuma combinacao de itens coube na mochila.\n");
```

```c
    } else {
        printf("\nPeso total na mochila: %d (Capacidade: %d)\n", peso_atual, capacidade);
        printf("Valor total na mochila: %d\n", valor_total);
    }

    return 0;
}
```

2.3 Programação Dinâmica

```c
#include <stdio.h>

#define MAX_ITENS 100
#define MAX_PESO 1000

typedef struct {
    char nome[50];
    int peso;
    int valor;
} Item;

int tabela[MAX_ITENS + 1][MAX_PESO + 1]; // tabela para armazenar os valores máximos

// função recursiva
int mochilaRec(int i, int W, Item itens[]) {
    // caso base: sem itens ou capacidade zero
    if (i == 0 || W == 0)
        return 0;

    // se já foi calculado, retorna o valor armazenado
    if (tabela[i][W] != -1)
        return tabela[i][W];

    // verifica se o peso do item atual for maior que a capacidade restante
    if (itens[i - 1].peso > W)
        return tabela[i][W] = mochilaRec(i - 1, W, itens);


    // verifica se ele não pega o item (semItem) ou se pega o item (comItem)
    int semItem = mochilaRec(i - 1, W, itens);
    int comItem = itens[i - 1].valor + mochilaRec(i - 1, W - itens[i - 1].peso, itens);

    // escolhe o maior valor entre os dois
    tabela[i][W] = (comItem > semItem) ? comItem : semItem;
    return tabela[i][W];
}

// função para printar os itens escolhidos
void printarItensEscolhidos(int n, int W, Item itens[]) {
    printf("\nItens escolhidos:\n");
```

```c
    int w = W;
    for (int i = n; i > 0 && w > 0; i--) {
        // se o valor atual é diferente do de cima, o item foi escolhido
        if (tabela[i][w] != tabela[i - 1][w]) {
            printf(" - %s (Peso: %d, Valor: %d)\n", itens[i - 1].nome, itens[i - 1].peso, itens[i - 1].valor);
            // atualiza o peso restante da mochila
            w -= itens[i - 1].peso;
        }
    }
}

int main() {
    int n, capacidade;
    printf("Esse programa resolve o problema da mochila (knapsack problem) atraves da programação dinâmica.\n\n");

    printf("Informe o peso maximo suportado pela mochila: ");
    scanf("%d", &capacidade);

    printf("Digite quantos itens serao avaliados: ");
    scanf("%d", &n);

    Item itens[n];

    printf("Digite os %d itens no formato\n\"nome\" \"peso\" \"valor\":\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%s %d %d", itens[i].nome, &itens[i].peso, &itens[i].valor);
    }

    // inicializa tabela com -1
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= capacidade; j++)
            tabela[i][j] = -1;

    int resultado = mochilaRec(n, capacidade, itens);

    printf("\nValor maximo obtido: %d\n", resultado);


    printf("\n--- Itens Escolhidos pela Programação Dinâmica ---\n");
    printf("Itens na mochila:\n");
    printarItensEscolhidos(n, capacidade, itens);

    return 0;
}
```

3 Equações de Recorrência

3.1 Brute Force

Analisando o algoritmo, a função encontrar_melhor_combinacao decide para cada item, se ele deve ser incluido ou não na mochila.

3.1.1 Equação de Recorrência:

Analisando a função recursiva, podemos deduzir a equação de recorrência da recursão:

$$T(n) = 2T(n-1) + c$$

Isso porque, a cada item, a função faz 2 chamadas para o próximo item, uma incluindo o item atual, e outra não.

3.1.2 Condições base:

A condição base ocorre quando **indice == n_total_itens**, ou seja, quando todos os itens já foram considerados para uma determinada combinação.

3.1.3 Transições entre Subproblemas:

Para cada item, o problema é subdividido em 2 partes:

- **Não inclui o item:** Resolve o problema para o restante dos itens com a mesma capacidade da mochila.
- **Inclui o item:** Resolve o problema incluindo o item na mochila, e com a capacidade restante da mochila, escolhe os próximos itens.

3.2 Programação Dinâmica

Nesta abordagem, utilizamos **Memoização**, que consiste em guardar os valores já calculados de subproblemas, para utilizar em futuros cálculos, assim, sendo mais eficiente. O algoritmo implementado utiliza a função **mochilaRec(i, W)**, que calcula o valor máximo para os **i** primeiros itens, com uma capacidade **W** da mochila.

3.2.1 Equação de Recorrência:

A equação de recorrência que descreve o valor ótimo $V(i, w)$ para os i primeiros itens com capacidade w é definida pela decisão de incluir ou não o i-ésimo item:

$$V(i, w) = \begin{cases} V(i-1, w) & \text{se } p_i > w \\ \max(V(i-1, w), v_i + V(i-1, w - p_i)) & \text{se } p_i \leq w \end{cases}$$

3.2.2 Condições base:

A recursão para quando não há mais itens para considerar **(i = 0)** ou quando a capacidade da mochila é zero **(W = 0)**.

3.2.3 Transições entre Subproblemas:

Para cada item i e capacidade W, o problema da decisão é dividido em subproblemas menores:

- **Se o item i-1 não cabe na mochila (peso_i-1 > W):** O valor ótimo é o mesmo que o valor ótimo para i-1 itens com a mesma capacidade W.
- **Se o item i-1 cabe na mochila:** A solução será o máximo entre:
  - **Não incluir o item:** O valor é V(i-1, W).
  - **Incluir o item:** O valor é a soma do valor do item atual (valor_i-1) com o valor ótimo para i-1 itens e a capacidade restante (W - peso_i-1).

3.3 Algoritmo Guloso

Nesse caso, não utiliza uma equação de recorrência, já que o algoritmo realiza uma ordenação por razão/peso, e logo depois realiza uma escolha local, mas podemos analisar o quick sort, já que ele determina o comportamento do algoritmo.

3.3.1 Equação de Recorrência:

- **Caso médio:** O algoritmo particiona o vetor em 2 partes para organização.

$$T(n) = 2T(n/2) + O(n)$$

- **Pior caso:** O pivô escolhido divide o vetor em uma parte com n-1 elementos e outra com 0.

$$T(n) = T(n-1) + O(n)$$

3.3.2 Condições base:

A recursão do quicksort para quando uma partição tem menos de dois elementos (inicio >= fim).

3.3.3 Transições entre Subproblemas:

O problema de ordenar n elementos é reduzido a ordenar duas sub-partições menores após posicionar o pivô corretamente.

4 Análise de Complexidade Temporal

4.1 Brute Force

Utilizando o método de expansão para resolver a recorrência $T(n) = 2T(n-1)+c$, temos:

$$\begin{aligned} T(n) &= 2T(n-1) + c \\ &= 2(2T(n-2) + c) + c = 2^2T(n-2) + 2c + c \\ &\vdots \\ &= 2^kT(n-k) + c\sum_{i=0}^{k-1} 2^i \end{aligned}$$

Com a condição base $T(0)$, temos $k = n$, resultando em uma complexidade de ordem $O(2^n)$.

4.2 Algoritmo Guloso

A complexidade do Quicksort no caso médio, $T(n) = 2T(n/2) + O(n)$, pode ser resolvida pelo Teorema Mestre:

- Parâmetros: $a = 2$, $b = 2$, $f(n) = O(n)$.
- Cálculo do expoente crítico: $\log_b a = \log_2 2 = 1$.
- Comparação: $f(n) = O(n)$ é da mesma ordem de $n^{\log_b a} = n^1$.
- Conclusão (Caso 2 do Teorema): $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n \log n)$.

4.3 Programação Dinâmica

A complexidade temporal da solução com programação dinâmica é determinada pelo custo de resolver a recorrência de valor $V(i, w)$ utilizando memoização.

A complexidade total é, portanto, o número total de estados possíveis multiplicado pelo custo constante para calcular cada um:

$$\begin{aligned} T(n, W) &= (\text{Número de subproblemas}) \times (\text{Custo por subproblema}) \\ &= (n \times W) \times O(1) \\ &= O(n \cdot W) \end{aligned}$$

O número de subproblemas é definido pelas variáveis da recorrência: i varia de 0 a n, e w varia de 0 a W. Isso cria uma tabela de $n \times W$ estados distintos a serem resolvidos.

5 Análise Empírica x Análise Teórica

5.1 Gráfico

Para a análise empírica, o tempo de execução dos três algoritmos (Força Bruta, Guloso e Programação Dinâmica) foi medido utilizando Python. Foram realizados testes para diferentes quantidades de itens (n), mantendo a capacidade da mochila sempre constante. Devido ao alto custo da Força Bruta, seus testes foram limitados a um máximo de 20 itens, enquanto os outros dois algoritmos foram testados com até 100 itens para demonstrar seu melhor desempenho.

Os dados coletados foram então utilizados para gerar um gráfico comparativo, que plota o número de itens (n) contra o tempo de execução. Para visualizar de forma eficaz a diferença entre as performances, foi aplicada uma escala logarítmica ao eixo do tempo, permitindo que o crescimento das curvas pudessem ser comparados claramente no mesmo gráfico.

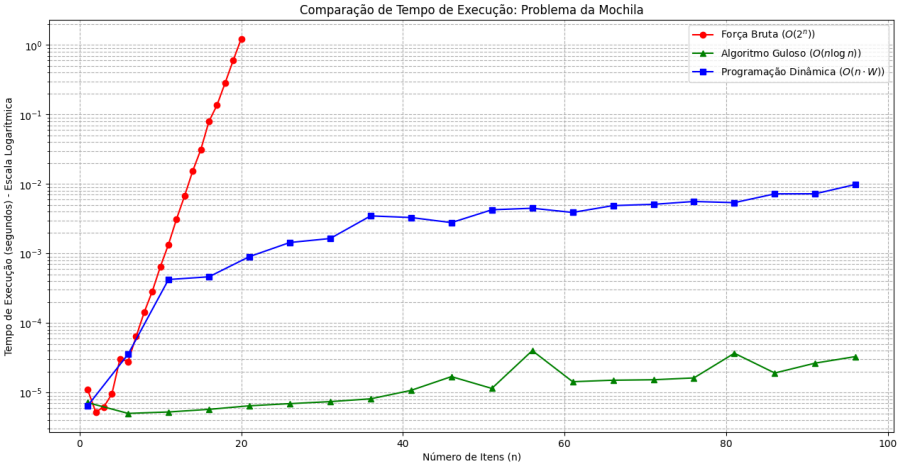

Figura 1: Figura comparando os tempos de execução

5.1.1 Brute Force

A linha vermelha no gráfico mostra um crescimento extremamente rápido, mesmo para valores pequenos de n. O tempo de execução praticamente dobra a cada novo item adicionado. Essa curva de crescimento explosivo corresponde à complexidade teórica exponencial de $O(2^n)$. Na prática, o algoritmo se torna inviável para n maior que 20.

5.1.2 Algoritmo Guloso

A linha verde mostra um crescimento muito baixo. O tempo é dominado pela ordenação inicial dos itens (Quick Sort). A curva se assemelha a uma linha quase reta em escala linear, o que é consistente com a complexidade quase-linear de $O(n \log(n))$. É o algoritmo mais eficiente.

5.1.3 Programação Dinâmica

A linha azul também exibe um crescimento muito mais controlado em comparação com o Brute Force. Seu crescimento parece ser linear em relação a n, pois a capacidade W da mochila foi mantida constante nos testes. Se aumentássemos W junto com n, veríamos um crescimento maior. A análise empírica confirma que esta abordagem é muito mais eficiente que o Brute Force e garante a solução ótima, embora seja um pouco mais lenta que o algoritmo Guloso (que não garante a solução ótima).